PRODUCT REQUIREMENT DOCUMENT (PRD)

UMHackathon 2026

umhackathon@um.edu.my

Table of Contents

1. Project Overview 3
2. Background & Business Objective 3
3. Product Purpose 4
4. System Functionalities 4
5. User Stories & Use Cases 5
6. Features Included (Scope Definition) 6
7. Features Not Included (Scope Control) 6
8. Assumptions & Constraints 6
9. Risks & Questions Throughout Development 7

Project Name: SentimentFlow AI

Version: 1.0

1. Project Overview

- **Problem Statement:** Businesses face a chaotic influx of unstructured customer data, such as reviews, messages, and support tickets, which traditionally requires tedious manual review for every single interaction.
- **Target Domain:** Customer service automation, task management, and business intelligence.
- **Proposed Solution Summary:** SentimentFlow is a cutting-edge platform that bridges the gap between raw user feedback and actionable intelligence. It acts as a multi-stage pipeline where an AI "Brain" digests content to extract emotion, intent, and urgency, transforming chaos into a structured, prioritized queue of tasks and automated case summaries.

2. Background & Business Objective

- **Background of the Problem:** Currently, evaluating customer sentiment and urgency across scattered channels relies heavily on manual labor, making it difficult to maintain a scalable support infrastructure.
- **Importance of Solving This Issue:** By automating the categorization and prioritization of incoming messages, businesses can drastically reduce their manual workload and ensure that critical issues are addressed immediately.
- **Strategic Fit / Impact:** SentimentFlow aligns with advanced Large Language Model (LLM) ecosystems by seamlessly integrating Google's Gemini AI to generate insights, freeing up authorized personnel to review AI decisions rather than start from scratch.

3. Product Purpose

- **3.1. Main Goal of the System**
  - To automate and optimize customer service workflows by combining deep sentiment analysis with intelligent task management.

- **3.2. Intended Users (Target Audience):**
  - Administrators / Authorized Personnel
  - Merchants (via the Merchant Dashboard)
  - Customers (via the Customer Portal)

4. System Functionalities

- **4.1 Description**
  - The system uses a FastAPI backend and a Streamlit frontend to process unstructured conversational data, dynamically categorize the data in a SQLite database, and display it on a real-time monitor.

- **4.2 Key Functionalities:**
  - **Advanced Sentiment Analysis:** Utilizes AI to go beyond basic positive/negative tagging by detecting specific emotions such as anger, frustration, or joy.
  - **Intelligent Case Management:** Auto-generates concise case summaries, suggests appropriate actions, and prioritizes tickets.
  - **Real-time Monitoring:** Provides a live dashboard visualizing system health, sentiment trends, and feedback volume.
  - **Secure Admin Interface:** A protected portal for authorized personnel to manage workflows, audit past actions, and ensure data integrity.

- **4.3 AI Model & Prompt Design:**
  - **4.3.1. Model Selection:** The system utilizes the gemini-3-flash-preview model via the Google genai SDK for its core reasoning engine. This model was chosen for its capability to rapidly dissect content and extract structured data like sentiment scores and urgency levels.
  - **4.3.2. Prompting Strategy:** The system employs structured Pydantic schemas (e.g., DataSchema, UnifiedResponseSchema, FeedbackAnalysisSchema) to force the LLM to return strictly formatted data containing sentiment, suggested actions, and verification details.
  - **4.3.3. Context & Input Handling:** User messages are passed through a ProcessRequest API endpoint, which can accept unstructured messages alongside optional context like customer_id and order_id to aid the classification process.
  - **4.3.4. Fallback & Failure Behavior:** The AI engine is designed to return an is_ambiguous boolean flag and a VerificationSchema (containing status and detail) to handle uncertainty or unclear user intents.

5. User Stories & Use Cases

- **User Stories:**
  - "As a business owner, I want a live dashboard to visualize sentiment trends and incoming feedback volume so I can instantly stay connected with my customer base."
  - "As an administrator, I want the system to suggest appropriate resolutions for incoming tickets so I can drastically reduce my manual workload."
- **Use Cases (Main Interactions):**
  - **Message Processing Flow:** A user submits a message via the Customer Portal. The FastAPI backend routes it to the SentimentFlowEngine, which classifies it and either returns an FAQ or creates a new support ticket in the database.
  - **Dashboard Resolution Flow:** A merchant logs into the secure Merchant Dashboard. They review the active complaints queue and click to update the ticket status (e.g., triggering a patch request to 'Approve & Resolve' the ticket).

6. Features Included (Scope Definition)

- Streamlit-based frontend with a custom CSS premium dark-mode aesthetic.
- Session state routing between a "Customer Portal" and a "Merchant Dashboard".
- A centralized FastAPI backend featuring distinct endpoints for processing messages, updating tickets, submitting feedback, and managing Todos.
- Local data persistence using SQLite, encompassing dedicated tables for Tickets, Feedback, Orders, Todos, and a Knowledge Base.

7. Features Not Included (Scope Control)

- External/Cloud Database implementations (The system relies strictly on a local sentimentflow.db SQLite file initialized on module load).
- Complex multi-factor authentication (The system currently relies on a basic boolean is_authenticated session state flag).

8. Assumptions & Constraints

- **Technical Constraints:** The project operates with hardcoded local environment variables, assuming the FastAPI backend is running locally at http://localhost:8000. It also requires a Python virtual environment to manage dependencies like uvicorn and pydantic.
- **System Input Constraints:** The Streamlit frontend server must be run concurrently with the Uvicorn backend server in the background for full functionality.
- **API Assumptions:** The environment assumes the presence of a valid .env file containing a GEMINI_API_KEY for the AI processing to function.

9. Risks & Questions Throughout Development

- **Data Structure Rigidity:** What happens if the Gemini model fails to properly adhere to the required Pydantic UnifiedResponseSchema during edge-case queries?
- **Thread Safety:** Since the application uses SQLite with check_same_thread=False, how will the database handle potential write locks if multiple customer portal users submit feedback simultaneously?